

Preparation of Papers for IEEE ACCESS

FIRST A. AUTHOR¹, (Fellow, IEEE), SECOND B. AUTHOR², AND THIRD C. AUTHOR, Jr.³, (Member, IEEE)

¹National Institute of Standards and Technology, Boulder, CO 80305 USA (e-mail: author@boulder.nist.gov)

²Department of Physics, Colorado State University, Fort Collins, CO 80523 USA (e-mail: author@lamar.colostate.edu)

³Electrical Engineering Department, University of Colorado, Boulder, CO 80309 USA

Corresponding author: First A. Author (e-mail: author@boulder.nist.gov).

This paragraph of the first footnote will contain support information, including sponsor and financial support acknowledgment. For example, "This work was supported in part by the U.S. Department of Commerce under Grant BS123456."

ABSTRACT Current development of microservice architectures heavily relies on API gateways and high-performance RPC frameworks like gRPC, the implementation details of the inter-service communication layer are often abstracted away in system design. Our paper empirically investigates the performance degradation caused by application-layer communication bottlenecks, with particular focus on gRPC connection management. We present a mathematical queueing model and algorithmic formalization to quantify the overhead of the connection-per-request (CPR) compared to a persistent channel reuse (PCR). Our evaluation results in Kubernetes environment tests show that converting CPR to PCR models can significantly reduce median latency by 97%—from over 7,000 ms to 150 ms under peak load. This change significantly eliminates the performance bottleneck that caused an increase in end-to-end user experience latency time.

INDEX TERMS Cloud-Native, Connection Per Request, gRPC API, High Performance Computing, Kubernetes, Microservices, Persistent Channel Reuse

I. INTRODUCTION

THE transition to cloud-native microservices has established API gateways and high-performance RPC frameworks like gRPC as standard infrastructure. In this architecture, the API gateway functions as the singular entry point for external traffic, acting primarily as a reverse proxy and protocol transcoder. While external clients typically communicate using standard HTTP/1.1 and JSON payloads—often following RESTful paradigms as discussed in [1]—internal microservices increasingly rely on gRPC over HTTP/2 for high-performance communication [2]. Unlike the stateless nature of traditional REST, gRPC is specifically designed to multiplex many requests over a single persistent HTTP/2 connection. However, the implementation details of this inter-service communication layer are often abstracted away by frameworks. As study by [3], have exposed significant infrastructural overheads hidden within service mesh sidecars, we observe that analogous, undocumented performance penalties exist within the application layer of API gateways when protocol translations are mismanaged.

The specific issue addressed in this paper is the performance degradation caused by the "connection-per-request" (CPR) anti-pattern within these gateways. Previous

studies have shown how TCP and TLS handshake delays reduce throughput in distributed systems [4]. However, the direct application of legacy REST-based connection management to gRPC environments remains a pervasive flaw. We define gRPC connection churn as the continuous cycle of establishing and tearing down complex network state (TCP handshakes and HTTP/2 protocol negotiations) for every individual application-layer request, rather than multiplexing traffic. Establishing a new connection per request is an $O(N)$ operation, where N represents the number of incoming client requests. This directly conflicts with the $O(1)$ multiplexed design of HTTP/2, forcing gateway worker threads to spend excessive CPU cycles on network context switching. We explore this vulnerability using a standard Python WSGI stack [9] (Flask and Unicorn) as a representative testbed, as its synchronous worker model clearly exposes the severe latency penalties of blocking network I/O.

While the research community has modeled microservice performance extensively, the specific intersection of application-layer connection churn and queueing network saturation remains empirically unquantified. Recent academic works, such as those by Pinheiro et al. [5], utilize stochastic Petri Nets to evaluate microservice resilience; however, evaluating latency under load requires a shift toward queueing

theory. We argue that bottlenecks at the implementation level, particularly the phenomenon of gRPC connection churn, lead to a dynamic increase in the service time (S) of the system. In queueing theory [6], an inflated service time severely degrades the theoretical service rate (μ). This churn converts what should be a constant-time RPC execution into a highly variable, load-dependent event. Previous research has not addressed how this specific application-layer mismanagement forces premature queue saturation long before physical CPU or memory limits are reached.

Therefore, this paper demonstrates how converting the original CPR approach into a Persistent Channel Reuse (PCR) approach, as shown in the Fig. ??—by refactoring the gateway code to initialize a global, thread-safe gRPC channel at startup—eliminates this bottleneck. To validate this, the specific contributions of this paper are as follows:

- We model the request patterns and application-layer logic based on both the CPR and the refactored PCR approaches.
- We introduce a queueing-theoretic model to explain the non-linear latency spikes observed under high load, linking application-layer connection churn to queue saturation.
- We benchmark these models within an actual Kubernetes environment using controlled isolation environment—ensuring the gateway is evaluated independently of backend database or business logic variance—to validate the correctness of the theoretical model against empirical runs.

II. MATHEMATICAL MODELING OF GRPC CONNECTION MANAGEMENT

In order to comprehend the communication bottleneck, it is essential to break down the entire lifecycle of a request as it progresses through the API gateway. This analysis will help us identify the key stages and factors contributing to the bottleneck in the communication process. We model this environment by evaluating two distinct application gateway architectures: a synchronous model (e.g., Flask thread pools) and an asynchronous model (e.g., FastAPI event loops).

Let T_{total} be the total end-to-end latency experienced by the client. We define T_{total} as:

$$T_{total} = T_{http} + W_{queue} + S_{gateway} + T_{network} \quad (1)$$

Where:

T_{http} : ingress HTTP processing time.

W_{queue} : queueing delay at the gateway's worker processes.

$S_{gateway}$: service time required by the gateway to establish and execute the downstream gRPC call.

$T_{network}$: physical network propagation delay and backend computation time (baseline constant $c \approx 3$ ms).

The critical variable in our study is $S_{gateway}$. We model this variable under two distinct operational paradigms, representing the system state before and after the autonomic controller's intervention.

A. PARADIGM 1: CONNECTION-PER-REQUEST (CPR)

In the Connection-Per-Request (CPR) anti-pattern, as shown in the CPR section of Fig. 1, the application gateway initializes a new gRPC channel for every incoming request. The service time S_{cpr} is defined as:

$$S_{cpr} = t_{tcp} + t_{http2} + t_{rpc} + t_{teardown} \quad (2)$$

Where t_{tcp} is the TCP 3-way handshake, t_{http2} is the HTTP/2 SETTINGS frame negotiation, t_{rpc} is the actual payload serialization/transmission, and $t_{teardown}$ is the TCP teardown (FIN/ACK). Because t_{tcp} and t_{http2} require multiple network round-trip times (RTTs) and significant CPU context switching, $S_{cpr} \gg t_{rpc}$.

In decentralized cloud-native architectures, the Client-side Proxy Routing (CPR) model dictates that the primary application container natively executes its own network routing and load balancing (e.g., utilizing native gRPC load balancing). This proxyless architecture minimizes intermediate network hops, optimizing baseline latency and reducing steady-state resource consumption. However, previous studies on proxyless service meshes demonstrate a critical vulnerability: the CPR model tightly couples the computational domain with the networking domain.

From a queueing theory perspective, the CPR model operates as a rigid $M/M/c$ queue. Under acute traffic surges, the network routing logic aggressively competes with the core application logic for limited CPU cycles and thread pools. When the system arrival rate approaches or exceeds the mathematical processing capacity ($\lambda \geq \lambda_{sat}$), the application's internal queue saturates rapidly, inevitably leading to dropped connections, unbounded tail latency, and catastrophic node failure.

B. PARADIGM 2: PERSISTENT CHANNEL REUSE (PCR)

In the optimized Persistent Channel Reuse (PCR) pattern, as shown in the PCR sections of Fig. 1 for both the synchronous and asynchronous gateway models, the architecture initializes a single, thread-safe gRPC channel during application startup. All incoming requests are multiplexed over this channel. The setup costs are amortized over millions of requests, approaching zero per request. Thus, the service time S_{pcr} simplifies to:

$$S_{pcr} = t_{rpc} \quad (3)$$

The PCR model completely decouples the data plane from the application by offloading routing, load balancing, and telemetry to a dedicated, out-of-process sidecar proxy (Envoy). This structural shift is dynamically enacted by the autonomic controller when saturation is detected.

While conventional research often criticizes the PCR model for introducing a continuous “service mesh tax”—permanent CPU, memory, and latency overheads during normal operations—its architectural strength lies in robust queue management. In the PCR model, the

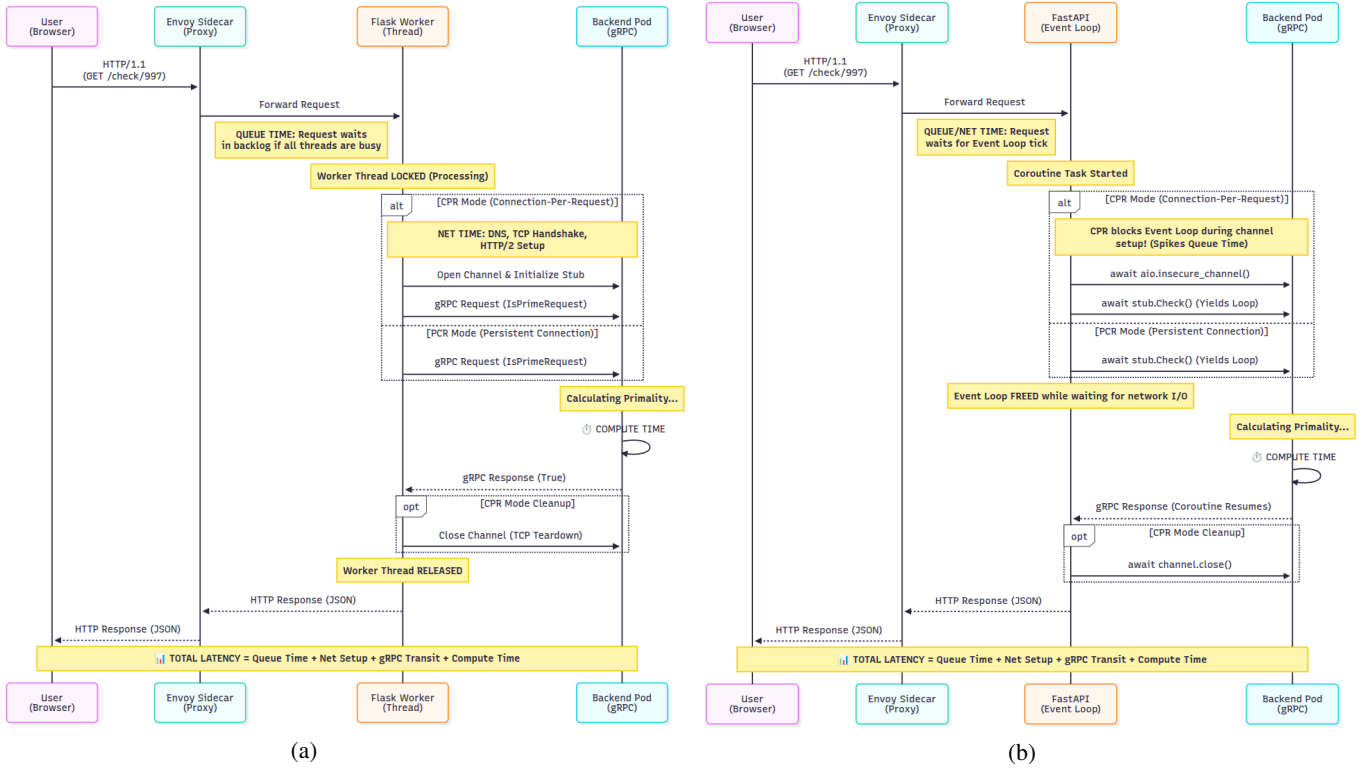


FIGURE 1. Sequence process of synchronous vs asynchronous communication models of cloud-native application on Connection Per Use (CPR) and Persistent Connections Reuse (PCR): (a) Synchronous CPR/PCR (Flask/Gunicorn); (b) Asynchronous CPR/PCR (Flask/Uvicorn).

injected sidecar acts as a highly optimized, asynchronous buffer. It absorbs massive traffic spikes, manages connection backlogs, and applies intelligent backpressure, thereby protecting the underlying application container (whether running synchronous threads or an asynchronous event loop) from localized saturation.

C. QUEUEING THEORY AND NON-LINEAR SATURATION

The catastrophic latency spikes observed in empirical testing cannot be explained by relying only on S_{cpr} value. We apply an $M/M/c$ queueing model, where incoming requests arrive at rate λ and are serviced by c gateway computational resources at rate $\mu = 1/S_{gateway}$.

The utilization of the gateway system is $\rho = \frac{\lambda}{c \cdot \mu}$. According to Little's Law [7] and standard queueing theory, the average queuing delay W_{queue} grows asymptotically as utilization $\rho \rightarrow 1$:

$$W_{queue} \propto \frac{\rho}{1 - \rho} \quad (4)$$

Because S_{cpr} is significantly larger than S_{pcr} , the service rate μ_{cpr} is artificially low. Therefore, under the CPR paradigm, the system reaches critical utilization ($\rho \rightarrow 1$) at a much lower arrival rate λ_{sat} . When λ exceeds this threshold, W_{queue} approaches infinity, resulting in the massive latency spikes and connection timeouts observed in our baseline data.

To fully contextualize the performance disparity between the CPR and PCR approaches, it is necessary to distinguish

the roles of the TCP session, the HTTP/2 stream, and application-layer channel management. A **TCP session** represents the underlying Layer-4 network socket, which incurs significant latency overhead to establish. An **HTTP/2 stream** is a lightweight, logical sequence of frames operating within that single TCP session; this is the native mechanism gRPC uses to multiplex thousands of concurrent requests over one connection.

The essential flaw of the CPR anti-pattern is that it actively circumvents this multiplexing capability. By instantiating and tearing down a gRPC channel within the local request handler, CPR forces the underlying HTTP/2 library to establish and destroy a unique TCP session for every single request, limiting multiplexing to a single stream per connection. Conversely, the PCR approach, enabled by the Envoy proxy, does not rebuild HTTP/2 features in the application layer. It is a structural design pattern that initializes a single, global TCP session, keeping the L4 connection persistent so that the native L7 HTTP/2 stream multiplexing functions as intended, seamlessly absorbing the load that causes the CPR model to fail.

D. FRAMEWORK-SPECIFIC QUEUEING DYNAMICS

While the general queueing delay W_{queue} reflects system-level saturation, the structural behavior under high concurrency diverges fundamentally depending on the application framework's concurrency model. We distinguish between the multi-threaded synchronous model and the single-threaded

asynchronous event-loop model.

1) Synchronous Thread-Pool Model ($M/M/c$)

In the synchronous architecture (e.g., Flask), the system relies on a thread pool of c parallel worker processes. Each worker handles a single connection sequentially. When a request initiates an un-multiplexed gRPC call under the CPR paradigm, the worker thread is blocked for the entire duration of S_{cpr} :

$$T_{\text{blocked}} = t_{\text{tcp}} + t_{\text{http2}} + t_{\text{rpc}} + t_{\text{teardown}} \quad (5)$$

When the arrival rate exceeds the aggregate thread processing capacity ($\lambda \geq c\mu_{cpr}$), all c workers become simultaneously occupied in I/O waiting states. Subsequent requests are diverted to the OS-level TCP listen backlog. Once this backlog fills, incoming SYN packets are dropped, driving $W_{\text{queue}} \rightarrow \infty$ and causing immediate client-side connection timeouts.

2) Asynchronous Event-Loop Model ($M/G/1 - PS$)

In contrast, the asynchronous architecture (e.g., FastAPI) executes over an event loop using non-blocking I/O primitives. Rather than modeling the system as c discrete servers, it is more accurately characterized as an $M/G/1$ Processor Sharing (PS) queue, where a single execution thread distributes CPU time across N_{active} concurrent tasks.

When an asynchronous request initiates a gRPC call, it registers an I/O callback and yields control back to the event loop during the latency window $t_{\text{tcp}} + t_{\text{http2}}$. The service time distribution G is general rather than exponential due to asynchronous context switching. The average queueing delay under Processor Sharing is expressed as:

$$W_{\text{queue},PS} = \frac{\rho}{1 - \rho} \cdot S_g \quad (6)$$

Because the event loop does not block physical threads during network negotiation, the effective capacity limit is decoupled from OS thread count and is instead constrained by event-loop tick duration and memory overhead. This enables the asynchronous gateway to hold thousands of partially completed connections in a pending state, delaying catastrophic packet drops far beyond the limits of the synchronous $M/M/c$ model.

III. AUTONOMIC CONTROLLER

A. SENSING MECHANISM AND TELEMETRY ACQUISITION

The average service time, represented as S_{cpr} , serves as the fundamental calculation component in establishing the mathematical saturation limit of the autonomic controller. The controller relies on two complementary Prometheus telemetry metrics to accurately calculate the S_{cpr} variable in a distributed environment (e.g., RKE cluster). The controller continuously evaluates both metrics for further analysis: $grpc_client_roundtrip_latency_seconds_sum$ ($gRPC_{\text{sum}}$) and $grpc_client_roundtrip_latency_seconds_count$ ($gRPC_{\text{count}}$).

On the one hand, the $gRPC_{\text{sum}}$ metric serves as a continuous accumulator, gradually increasing over time. It tracks the absolute total seconds spent processing and responding to incoming remote procedure calls within the application. On the other hand, the $gRPC_{\text{count}}$ metric acts as a straightforward digital counter, registering the exact volume of discrete requests that have successfully completed their lifecycle.

To derive the real-time average service time for the control plane, the autonomic controller calculates the quotient of the rates of change of these two metrics over a sliding observation window. Mathematically, this relationship is expressed as the ratio of their respective variations over a specific time delta Δt :

$$S_{cpr} = \frac{\Delta gRPC_{\text{sum}} \Delta t}{\Delta gRPC_{\text{count}} \Delta t} \quad (7)$$

In the field of classical queuing theory, service time (S) is commonly used to represent the duration a physical or virtual server is actively engaged in a single unit of work. By utilizing native gRPC roundtrip metrics, the controller captures highly accurate, comprehensive processing measurements instead of isolated internal application timers. This duration inherently includes data serialization, framework event-loop overhead, and the core CPU execution required for the prime number calculations. These processes are illustrated in the Fig. 2.

B. CALCULATION AND ANALYSIS

1) Predictive Mathematical Formulation:

To mathematically formalize the system's performance under increasing target rates, the system architecture is modeled as an $M/M/c$ queue. This assumes a Poisson arrival process for incoming requests with rate λ (target rate), exponentially distributed service times with rate μ (where $\mu = 1/S_{cpr}$), and c parallel computing resources [12] [13]. For the system to remain stable, utilization must remain below capacity ($\rho < 1$). As the target rate λ increases toward the total service capacity $c\mu$, the system enters a state of high utilization. According to Erlang's C formula, the probability that an arriving request must wait in the queue (P_Q) is:

$$P_Q = \frac{\frac{(c\rho)^c}{c!(1-\rho)}}{\sum_{n=0}^{c-1} \frac{(c\rho)^n}{n!} + \frac{(c\rho)^c}{c!(1-\rho)}} \quad (8)$$

Consequently, the average expected wait time in the queue (W_{queue}) grows non-linearly:

$$W_{\text{queue}} = \frac{P_Q}{c\mu(1-\rho)}$$

As $\rho \rightarrow 1$, W_{queue} approaches infinity. This mathematical phenomenon explains the severe spikes observed in empirical $p99$ tail latencies. Rather than waiting for system utilization to completely fail ($\rho \geq 1.0$), the controller continuously polls real-time telemetry to compute the empirical service time (S_{cpr})

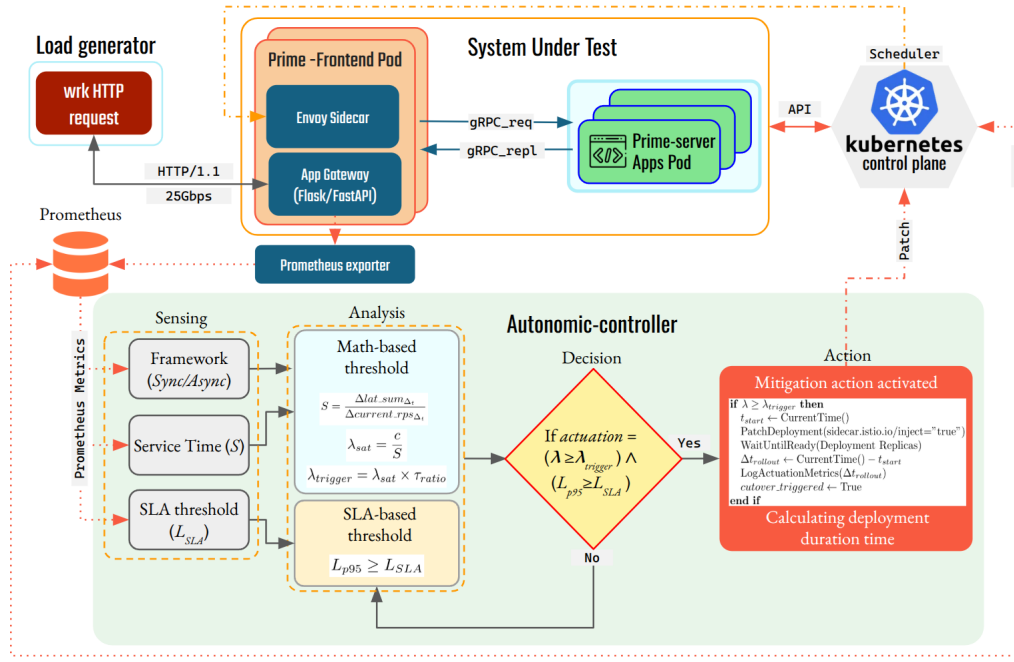


FIGURE 2. The autonomic controller architectures.

and calculates the mathematical saturation limit (λ_{sat}) based on the application's theoretical concurrency (c):

$$\lambda_{sat} = \frac{c}{S_{cpr}}$$

To guarantee survivability, the controller defines a predictive intervention boundary ($\lambda_{trigger}$) using a configurable safety threshold ratio (T_{ratio}):

$$\lambda_{trigger} = \lambda_{sat} \times T_{ratio}$$

2) SLA Evaluation and Bottleneck Verification:

While the predictive trigger anticipates resource exhaustion based on theoretical capacity, actual system degradation must be empirically verified to prevent premature actuation. The controller achieves this through real-time Service Level Agreement (SLA) monitoring, specifically targeting the 95th percentile application latency (L_{p95}). To acquire this metric accurately, the control plane queries the `grpc_client_roundtrip_latency_seconds_bucket` histogram exposed directly by the application runtime. By calculating the 95th quantile from the rate of change across these distribution buckets over a one-minute sliding window, the system extracts a stabilized, real-time measurement of the tail latency. Relying on this internal gRPC telemetry rather than external HTTP network timers is a critical architectural choice; it strictly isolates the application's true compute and event-loop processing delays from external network artifacts, such as TCP backlog saturation or service

mesh queuing. Targeting the 95th percentile ensures the controller focuses on the degraded experience of the slowest requests while filtering out extreme, transient outliers (like the 99th percentile) that are often caused by routine garbage collection or minor operating system context switching. This observed L_{p95} value is continuously evaluated against a rigid, configurable latency boundary (L_{SLA} , set to 35 ms). Within the hybrid controller, this SLA evaluation acts as the definitive proof of exhaustion. The mathematical capacity limit ($\lambda_{trigger}$) flags that the system *should* be saturated based on average service times, but the L_{p95} breach provides the empirical confirmation that the synchronous thread pool or asynchronous event loop has actually failed to process the current concurrent workload. By requiring this latency condition, the controller completely avoids false-positive mitigations during brief, harmless traffic bursts.

C. ACTUATION AND DYNAMIC CUTOVER

The control framework directly solves architectural inelasticity by bridging the CPR and PCR states dynamically. To prevent false positives caused by transient network jitter, the controller implements a strict dual-condition actuation gate. The dynamic cutover is only authorized if the predictive mathematical limit is breached *and* the empirical SLA is actively failing (exceeding the 35 ms threshold). The final condition is defined as:

$$\text{Trigger} \rightarrow \text{True} \quad \text{if} \quad (\lambda \geq \lambda_{trigger}) \wedge (L_{p95} > 35 \text{ ms})$$

Upon satisfying this dual-condition logic, the autonomic controller intervenes proactively. It initiates a dynamic patch to the frontend deployment within the Kubernetes control plane, applying the *sidecar.istio.io/inject="true"* annotation. This action forces a rolling restart, attaching the Envoy proxy to the workload in real-time. By dynamically shifting from a low-overhead CPR topology to a highly resilient PCR topology exclusively when these two thresholds intersect, the controller caps the connection bottleneck and neutralizes exponential queuing delay before catastrophic SLA violations occur.

Algorithm 1 Autonomic Controller with Math-based Predictive & SLA-Aware Dynamic Cutover

```

1: Inputs: Poll interval  $t_{poll}$ , Minimum traffic threshold  $\lambda_{min} = 10$ , Latency SLA  $L_{SLA} = 50$  ms
2: Initialize: cutover_triggered  $\leftarrow$  False
3: while true do
4:   Sleep for  $t_{poll}$  seconds
5:   if cutover_triggered = True then
6:     continue {Idling post-actuation}
7:   end if
8:   Fetch Real-time Telemetry
9:    $\lambda \leftarrow$  QueryPrometheus(Rate of Requests (RPS))
10:   $S_{cpr} \leftarrow$  QueryPrometheus(Average Service Time (s))
11:   $L_{p95} \leftarrow$  QueryPrometheus(App P95 Latency (ms))
12:   $n_{threads} \leftarrow$  QueryPrometheus(Active Threads)
13:  if  $\lambda < \lambda_{min}$  or Telemetry is Null then
14:    continue {Wait for stable load}
15:  end if
16:  Framework Auto-Detection & Recalibration
17:  if  $n_{threads} > 50$  then
18:    mode  $\leftarrow$  SYNC (Flask)
19:  else
20:    mode  $\leftarrow$  ASYNC (FastAPI)
21:  end if
22:   $c \leftarrow 1.0$  {Normalized parallel compute capacity}
23:   $\tau_{ratio} \leftarrow 0.80$  {Target utilization threshold}
24:  Calculate Queueing Saturation Threshold
25:   $\lambda_{sat} \leftarrow \frac{c}{S_{cpr}}$ 
26:   $\lambda_{trigger} \leftarrow \lambda_{sat} \times \tau_{ratio}$ 
27:  Dual-Condition Evaluation
28:  if  $\lambda \geq \lambda_{trigger}$  and  $L_{p95} \geq L_{SLA}$  then
29:     $t_{start} \leftarrow$  CurrentTime()
30:    PatchDeployment(sidecar.istio.io/inject="true")
31:    WaitUntilReady(Deployment Replicas)
32:     $\Delta t_{rollout} \leftarrow$  CurrentTime()  $- t_{start}$ 
33:    LogActuationMetrics( $\Delta t_{rollout}$ )
34:    cutover_triggered  $\leftarrow$  True
35:  end if
36: end while

```

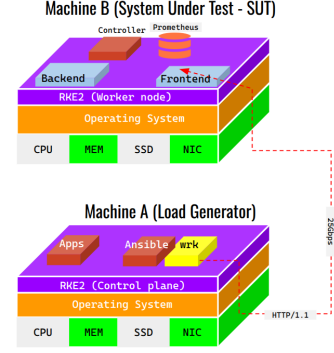


FIGURE 3. Kubernetes cluster environment.

IV. EXPERIMENT

To validate the proposed mathematical model and pinpoint accurately where latency overhead originates, we conducted controlled experiments in a containerized distributed setting. Our evaluation follows a strict component isolation strategy: we first measure end-to-end performance across the full system and then benchmark the backend service alone in order to distinguish computation time from communication-related delays. This was achieved by deploying a stateless, deterministic backend and enforcing Kubernetes Guaranteed QoS (matching CPU/memory requests to limits) to eliminate resource throttling and "noisy neighbor" interference.

Our testbed uses a Python WSGI stack (Flask/Gunicorn [8]) as a standard API gateway. We aim to check how gRPC connection churn impacts performance, not to compare frameworks. Synchronous threads are sensitive to blocking network inputs, highlighting queueing issues.

TABLE 1. Infrastructure specification

Components	Load generator (A)	SUT (B)
CPU	Intel Xeon E-2274G (4 cores)	AMD EPYC 9655P (96 cores)
Memory	32GB	194GB
Storage	SSD 1TB	SSD 1TB
Network	Mellanox MT27800 25Gb	Mellanox MT27710 25Gb
OS	Ubuntu 22.04 LTS	Ubuntu 24.04 LTS

A. EXPERIMENTAL ENVIRONMENT AND ARCHITECTURE

The experiment was conducted on a Kubernetes cluster composed of multiple worker nodes as shown in the Fig. 3. The system under test (SUT) consists of two main microservice components:

- **gRPC Backend Service (prime-server):** A high-performance Python application responsible for CPU-intensive prime number verification. To eliminate backend capacity as a confounding variable, this component was over-provisioned with 10 identical replica pods, deployed behind a Kubernetes NodePort Service.
- **API Gateway (prime-frontend):** A Python Flask application running within Gunicorn workers. This

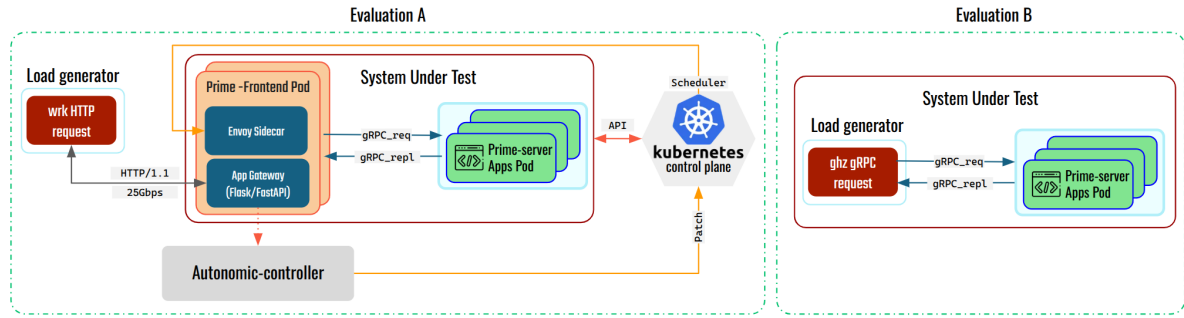


FIGURE 4. Evaluation scenario: (A) Comprehensive evaluations (end-to-end user experience); (B) Isolated backend service evaluation (compute time).

component acts as an HTTP/REST to gRPC proxy. It was strictly limited to 2 replica pods to intentionally force saturation and observe gateway-tier queuing dynamics under heavy load.

B. WORKLOAD GENERATION AND COMPONENT ISOLATION

To precisely calculate the application-layer gateway overhead, we utilized two specialized, open-source load-testing tools (*wrk2* [10] and *ghz* [11]), as shown in the Fig. ??:

- **Unoptimized Gateway (CPR):** Simulates Algorithm ?? (*mode* = CPR), forcing TCP/HTTP2 renegotiation for every incoming request.
- **Optimized Gateway (PCR):** Simulates Algorithm ?? (*mode* = PCR), routing concurrent accesses through a single, globally initialized gRPC channel.

Both tools executed 30 iterations for each target throughput level (ranging from 100 to 1000 Requests Per Second, or RPS), sustaining the load for 60 seconds per iteration to allow queuing buffers to stabilize.

V. RESULTS AND DISCUSSION

The empirical results have successfully proven that the predicted Asymptotic latency degradation is caused by the $M/M/c$ queuing model in the Connection-Per-Request (CPR) paradigm and have clearly demonstrated the transformational impact of Persistent Channel Reuse (PCR).

A. BASELINE: BACKEND GRPC PERFORMANCE

The *ghz* test results established an exceptional and highly stable baseline. Across the entire load spectrum (100 to 1000 RPS), the 10 prime-server backend pods consistently delivered a median (p50) latency of ~ 2.5 ms to 3.6 ms, with p90 latencies remaining firmly under 10 ms.

This flat latency curve definitively proved that the backend architecture was not the system bottleneck, as shown in the Fig. ?. By having 10 backend pods, we mathematically guaranteed that the backend possessed vastly more computational capacity than the frontend could ever saturate. The computational capacity of the prime number verification logic was highly resilient, isolating the source of any overarching performance degradation to the gateway tier.

Concurrency Configuration and Multiplexing: During empirical testing, both *ghz* and *wrk2* were configured to simulate high concurrency using a fixed pool of 50 simultaneous client connections across all target throughput levels (λ). It is critical to note how these concurrent accesses are handled internally by the optimized gateway. Under the PCR paradigm, the concurrent requests arriving at the gateway are not queued sequentially over the network; instead, the underlying gRPC framework maps these concurrent application-level accesses into discrete, parallel HTTP/2 streams over a *single* shared TCP connection. This mechanism prevents connection exhaustion and directly validates our mathematical assumption that S_{pcr} approaches pure RPC execution time without queuing at the network socket layer.

B. UNOPTIMIZED SYSTEM: THE CPR BOTTLENECK

Testing the end-to-end system utilizing the CPR paradigm revealed a catastrophic bottleneck, as shown in the Fig. ??-A (upper side). At a moderate load of 400 RPS, the system maintained a median latency of ~ 400 ms. However, as throughput increased to 600 RPS, the system hit the critical utilization threshold ($\rho \rightarrow 1$).

Median latency exploded to $> 7,100$ ms, and p99 latency spiked past 14,000 ms. This exponential degradation perfectly aligns with the queueing theory model where service time (S_{cpr}) is artificially inflated by excessive TCP/HTTP2 handshake overheads for every transaction. The gateway workers were entirely occupied managing network state rather than processing business logic.

C. OPTIMIZED SYSTEM: PCR IMPLEMENTATION

By deploying the algorithmic optimization detailed in Section III (establishing a persistent, multiplexed gRPC channel at application startup - Algorithm ??), the performance profile of the system was qualitatively transformed.

At 600 RPS, the optimized system recorded a median latency of ~ 150 ms, as shown in the Fig. ??-B (lower side). This represents a $\sim 97.9\%$ reduction in end-to-end latency compared to the CPR paradigm. Furthermore, the system successfully sustained loads up to 800 RPS before demonstrating signs of stress, effectively doubling the

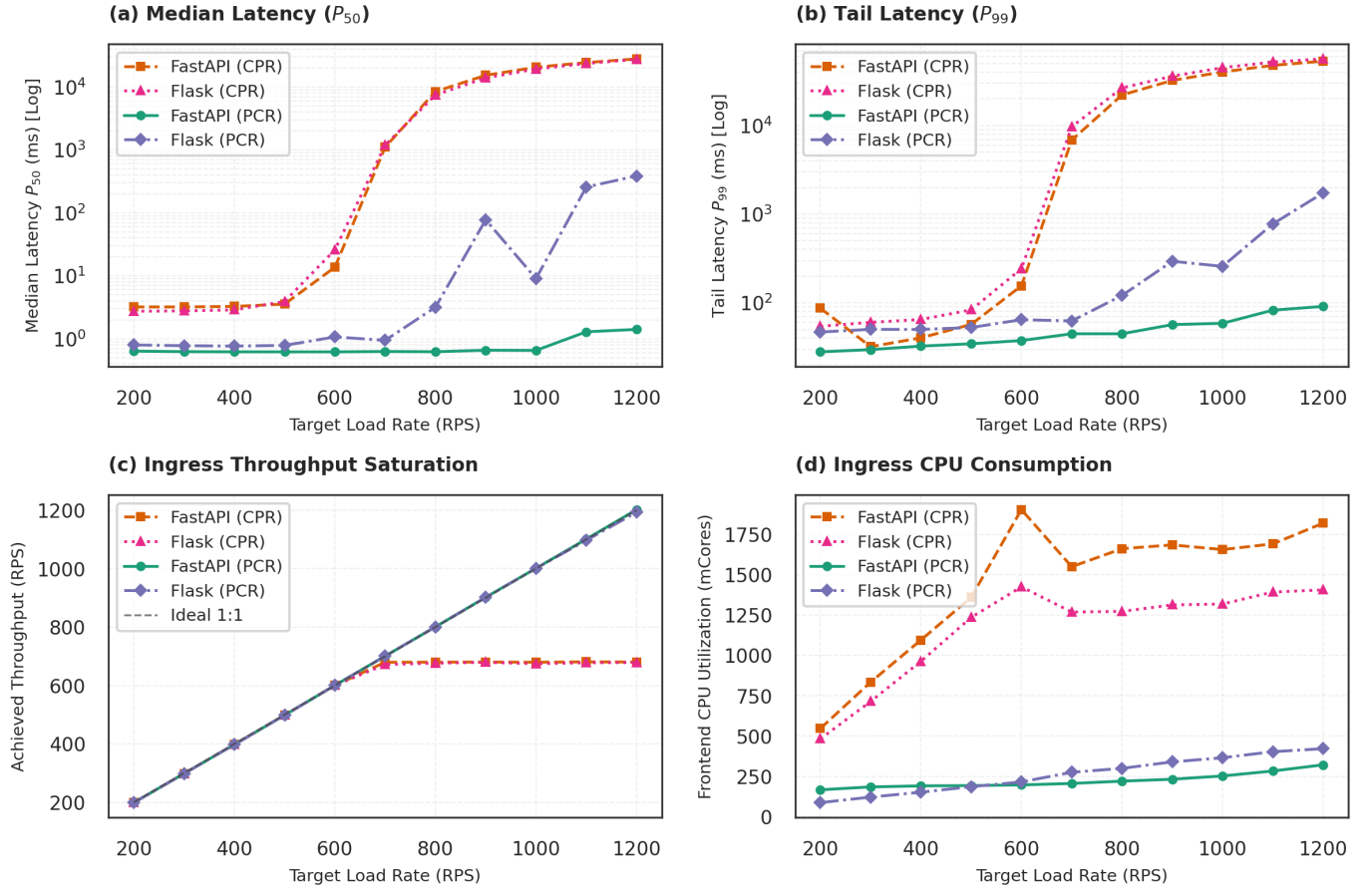


FIGURE 5. Baseline performance evaluation of frontend communication gateway (Synchronous and Asynchronous method) within Connection Per Request (CPR) and Persistent Connections Reuse (PCR) model: (a) median latency (p_{50}); (b) end-to-end latency (p_{99}); (c) systems throughput; (d) resource utilization (CPU).

system's maximum throughput capacity without provisioning any additional infrastructure resources.

D. QUANTIFYING THE INHERENT PROXY LATENCY GAP

By subtracting the baseline backend network latency ($T_{network}$) established via *ghz* from the full end-to-end system latency (T_{total}) measured via *wrk2*, we can isolate the underlying application-layer tax. We define this remaining overhead as the Inherent Proxy Latency Gap—the baseline latency strictly attributable to the framework's internal request handling, independent of network state management.

TABLE 2. Comparative Latency Overhead at Optimal Saturation Point (400 RPS)

Component level	Median latency (p_{50})	Percentage of total latency
Backend Only (<i>ghz</i>)	~ 2.8 ms	2.5%
Full System (CPR, Before)	~ 400 ms	100%
Full System (PCR, After)	~ 110 ms	100%
Gateway Overhead (PCR)	~ 107.2 ms	97.5%

As illustrated in Table 2, the results reveal a compelling paradox in the optimized PCR paradigm. Although the persistent channel implementation successfully averted queue

saturation and achieved a 97% reduction in total latency compared to the CPR baseline, a compositional analysis shows that the gateway tier still accounts for 97.5% of the remaining end-to-end response time.

This empirical gap directly connects our theoretical approach to the user experience. In our PCR model, the actual backend computational execution ($T_{network}$) requires only ~ 2.8 ms (2.5% of total time), while the remaining 107.2 ms is consumed by the gateway. Critically, this gap encompasses not only the Python WSGI [9] stack execution but also the protocol transcoding overhead—the computational cost of parsing incoming HTTP/1.1 JSON requests from *wrk2* and serializing them into HTTP/2 Protobuf messages for the gRPC backend.

This reveals a fundamental "Performance-Abstraction Paradox." While developers utilize API gateways to seamlessly bridge RESTful clients (HTTP/1.1) with high-performance microservices (gRPC), this protocol translation introduces a structural latency tax that dwarfs the business logic itself. Consequently, while our PCR optimization resolves protocol-level queue saturation, the absolute lower bound of system latency remains strictly dictated by the

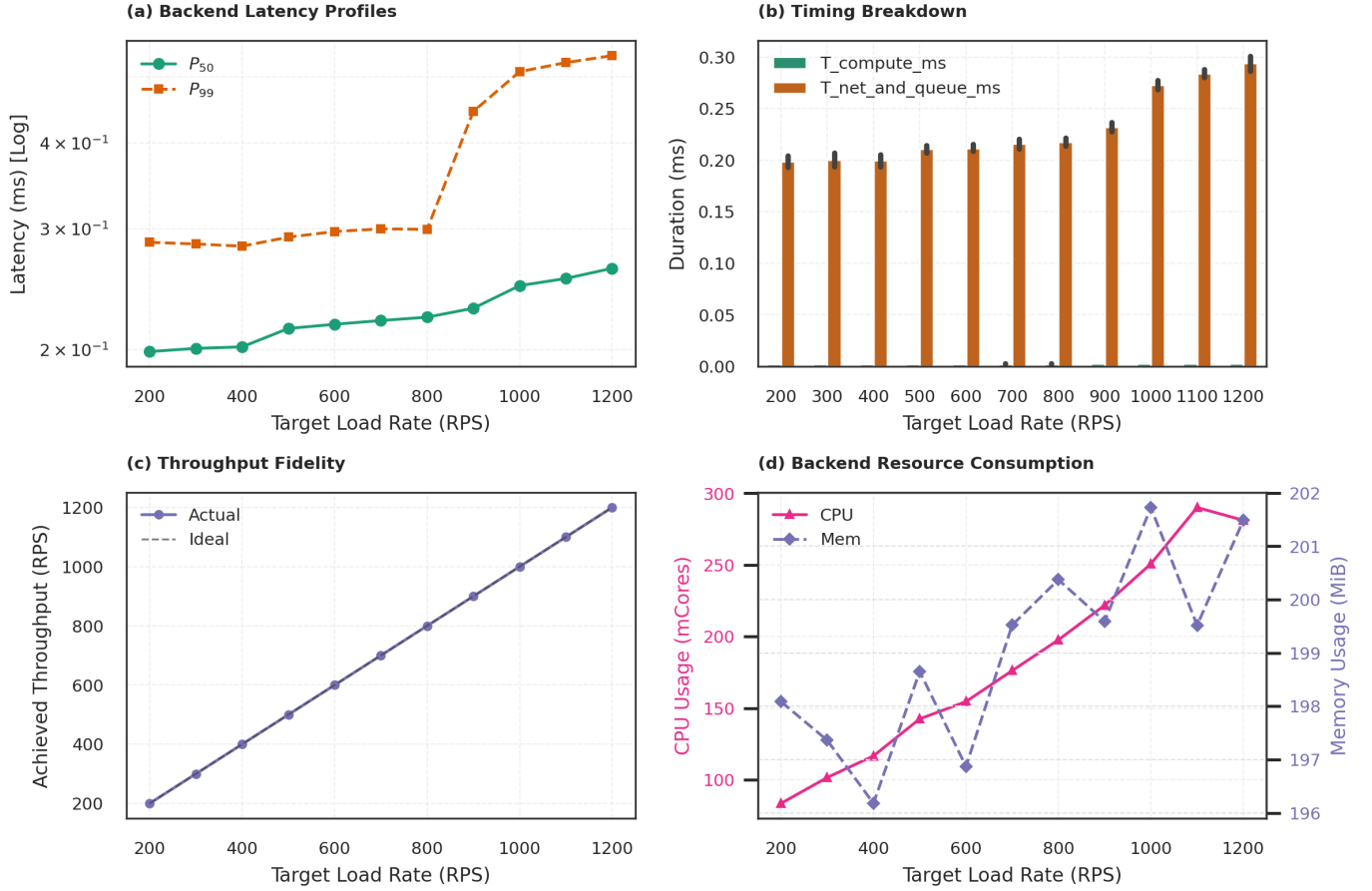


FIGURE 6. Baseline performance evaluation of backend service that evaluate directly to the backend server using ghz tool with gRPC load request: (a) median and tail latency (p_{50} , p_{99}); (b) compute vs net and queue time latency (t_{comp} , t_{net_queue}); (c) systems throughput; (d) resource utilization (CPU and Memory).

gateway's HTTP-to-gRPC transcoding efficiency.

E. EMPIRICAL VALIDATION OF THE QUEUEING MODEL

To rigorously evaluate the mathematical framework proposed in Section II, we mapped our empirical throughput and latency data to the $M/M/c$ queueing model. Our objective was to prove that the catastrophic latency spikes observed were not random system failures, but rather predictable queueing saturation events driven by connection churn.

In our system architecture, the gateway tier acts as the server pool (c workers). The arrival rate of incoming requests is defined as λ (varying from 100 to 1000 RPS). The service rate of the gateway, μ , is inversely proportional to the service time ($S_{gateway}$).

Under the Connection-Per-Request (CPR) paradigm, the service time (S_{cpr}) includes the mandatory TCP and HTTP/2 handshakes ($t_{tcp} + t_{http2}$). Because these network operations block gateway worker threads, μ_{cpr} is severely restricted.

Our empirical data aligns perfectly with the theoretical utilization formula:

$$\rho = \frac{\lambda}{c \cdot \mu}$$

According to Little's Law [7], queueing delay W_{queue} grows asymptotically as system utilization $\rho \rightarrow 1$:

$$W_{queue} \propto \frac{\rho}{1 - \rho}$$

Evaluating the CPR Saturation Point: During our baseline CPR testing, the system maintained a median latency of ~ 400 ms at $\lambda = 400$ RPS, indicating high but stable utilization ($\rho < 1$). However, when the arrival rate increased to $\lambda = 600$ RPS, the median latency violently spiked to $> 7,100$ ms. This exponential degradation confirms that the low service rate (μ_{cpr}) caused the gateway to cross the critical threshold of $\rho = 1$ at a mere 600 RPS. At this point, requests were arriving faster than connections could be negotiated, causing W_{queue} to theoretically approach infinity, which manifested empirically as 7-second tail latencies and connection timeouts.

Evaluating the PCR Optimization: By transitioning to the Persistent Channel Reuse (PCR) paradigm, we eliminated the handshake overhead, drastically reducing the service time to S_{pcr} and thereby significantly increasing the service rate (μ_{pcr}). Because $\mu_{pcr} \gg \mu_{cpr}$, the utilization ρ at the same arrival rate ($\lambda = 600$ RPS) remained well below 1.

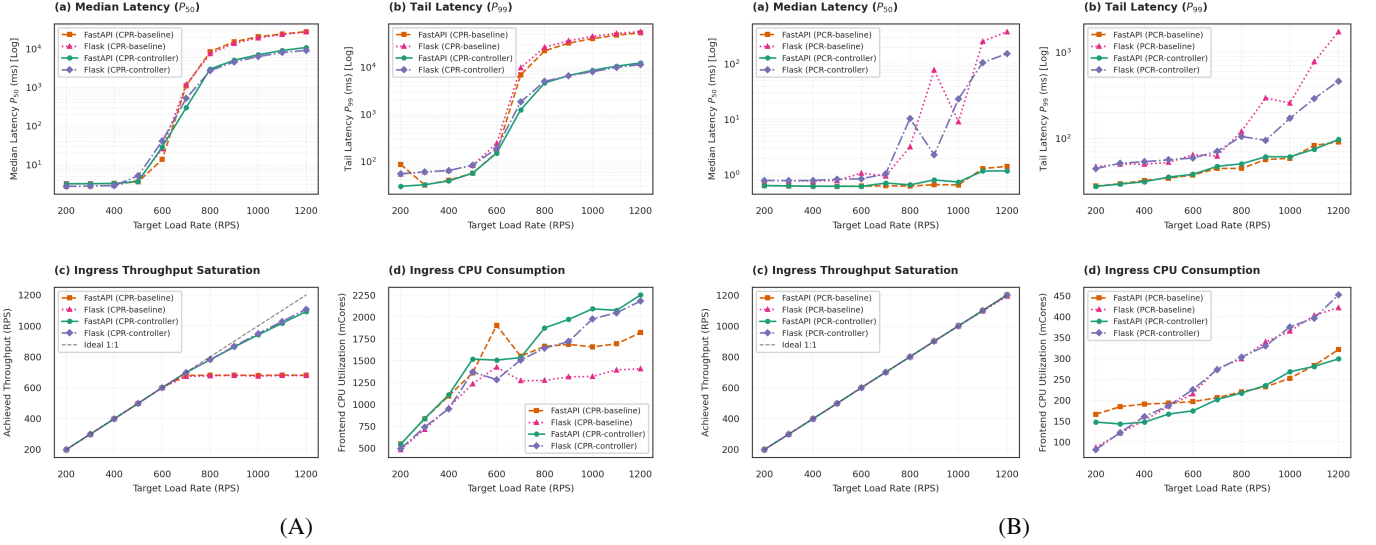


FIGURE 7. Baseline vs controller performance evaluation of frontend apps both in Synchronous and Asynchronous framework gateway within: (A) Connection Per-Request (CPR); (B) Persistent Connection Reuse (PCR) mode.

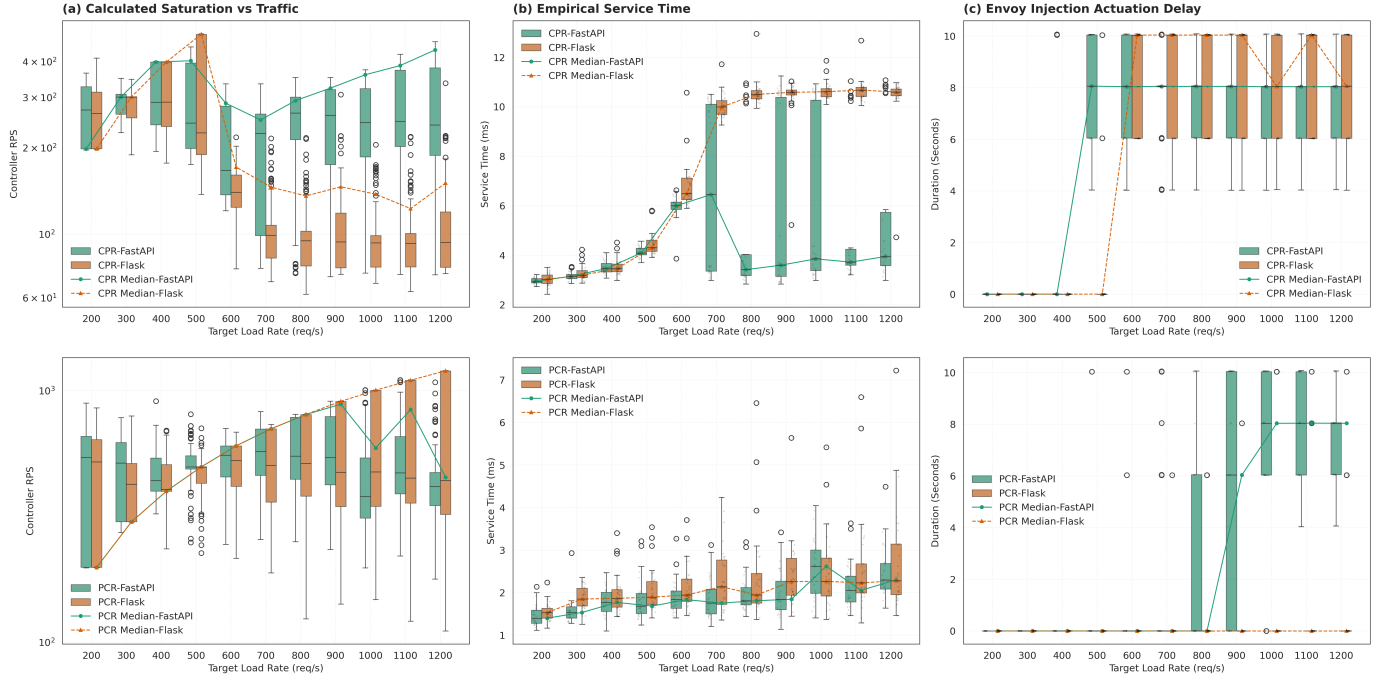


FIGURE 8. Autonomic controller performance evaluation in asynchronous (FastAPI/Uvicorn) and synchronous (Flask/Gunicorn) mode. Connection Per Request (CPR) - upper row, Persistent Connection Reuse (PCR) - lower row : (a) Calculated saturation point (λ_{sat}); (b) service time (S); (c) Actuation time when envoy deployed to frontend pods.

To clarify the protocol dynamics, the Persistent Channel Reuse (PCR) approach does not reimplement HTTP/2 features; rather, it maintains a single global TCP session to allow native HTTP/2 stream multiplexing to function as intended. Conversely, the CPR anti-pattern actively sabotages this multiplexing capability by forcing the underlying library to establish and destroy a unique TCP session for every individual request.

Consequently, the queueing delay W_{queue} collapsed. The

empirical result—a drop in latency from $> 7,100$ ms to exactly 150 ms at 600 RPS—serves as a direct physical validation of the $M/M/c$ model. It proves that correcting the $O(N)$ connection anti-pattern artificially scales the service rate μ , averting queue saturation without provisioning any additional hardware capacity (c).

VI. CONCLUSION AND FUTURE WORK

In this paper, we investigated the hidden performance costs of application-layer communication bottlenecks in microservice architectures, focusing on the gRPC connection churn flaw within API gateways. We modeled the request patterns of the "connection-per-request" (CPR) anti-pattern and contrasted it with a refactored Persistent Channel Reuse (PCR) approach. The observation of actual runs demonstrated that refactoring the gateway implementation to the PCR approach yielded a 97% reduction in median system latency—dropping from over 7,100 ms to 150 ms under a peak load of 600 RPS. This study proves that correcting application-layer protocol mismanagement successfully moves the performance bottleneck away from the infrastructure gateway and to the compute backend, highlighting that refactoring the gateway code is a critical aspect to scalability as the underlying architectural design.

Future work will expand this steady-state benchmarking methodology into the domain of Chaos Engineering. By introducing controlled faults (e.g., node resource starvation, sudden pod termination) under a persistent load.

VII. REFERENCE EXAMPLES

- *Basic format for books:*
J. K. Author, "Title of chapter in the book," in *Title of His Published Book*, xth ed. City of Publisher, (only U.S. State), Country: Abbrev. of Publisher, year, ch. x, sec. x, pp. xxx–xxx.
See [14], [15].
- *Basic format for periodicals:*
J. K. Author, "Name of paper," *Abbrev. Title of Periodical*, vol. x, no. x, pp. xxx–xxx, Abbrev. Month, year, DOI: 10.1109.XXX.123456.
See [16]–[18].
- *Basic format for reports:*
J. K. Author, "Title of report," Abbrev. Name of Co., City of Co., Abbrev. State, Country, Rep. xxx, year.
See [19], [20].
- *Basic format for handbooks:*
Name of Manual/Handbook, x ed., Abbrev. Name of Co., City of Co., Abbrev. State, Country, year, pp. xxx–xxx.
See [21], [22].
- *Basic format for books (when available online):*
J. K. Author, "Title of chapter in the book," in *Title of Published Book*, xth ed. City of Publisher, State, Country: Abbrev. of Publisher, year, ch. x, sec. x, pp. xxx–xxx. [Online]. Available: <http://www.web.com>
See [23]–[26].
- *Basic format for journals (when available online):*
J. K. Author, "Name of paper," *Abbrev. Title of Periodical*, vol. x, no. x, pp. xxx–xxx, Abbrev. Month, year. Accessed on: Month, Day, year, DOI: 10.1109.XXX.123456, [Online].
See [27]–[29].
- *Basic format for papers presented at conferences (when available online):*
J.K. Author. (year, month). Title. presented at abbrev. conference title. [Type of Medium]. Available: site/path/file
See [30].
- *Basic format for reports and handbooks (when available online):*
J. K. Author. "Title of report," Company. City, State, Country. Rep. no., (optional: vol./issue), Date. [Online] Available: site/path/file
See [31], [32].
- *Basic format for computer programs and electronic documents (when available online):*
Legislative body. Number of Congress, Session. (year, month day). *Number of bill or resolution, Title*. [Type of medium]. Available: site/path/file
See [33].
- *Basic format for patents (when available online):*
Name of the invention, by inventor's name. (year, month day). Patent Number [Type of medium]. Available: site/path/file
See [34].
- *Basic format for conference proceedings (published):*
J. K. Author, "Title of paper," in *Abbreviated Name of Conf.*, City of Conf., Abbrev. State (if given), Country, year, pp. xxxxx.
See [35].
- *Example for papers presented at conferences (unpublished):*
See [36].
- *Basic format for patents:*
J. K. Author, "Title of patent," U.S. Patent x xxx xxx, Abbrev. Month, day, year.
See [37].
- *Basic format for theses (M.S.) and dissertations (Ph.D.):*
 - 1) J. K. Author, "Title of thesis," M.S. thesis, Abbrev. Dept., Abbrev. Univ., City of Univ., Abbrev. State, year.
See [38], [39].
 - 2) J. K. Author, "Title of dissertation," Ph.D. dissertation, Abbrev. Dept., Abbrev. Univ., City of Univ., Abbrev. State, year.
See [38], [39].
- *Basic format for the most common types of unpublished references:*
 - 1) J. K. Author, private communication, Abbrev. Month, year.
 - 2) J. K. Author, "Title of paper," unpublished.
 - 3) J. K. Author, "Title of paper," to be published.
See [40]–[42].
- *Basic formats for standards:*
 - 1) *Title of Standard*, Standard number, date.
 - 2) *Title of Standard*, Standard number, Corporate author, location, date.
See [43], [44].
- *Article number in reference examples:*
See [45], [46].
- *Example when using et al.:*

See [47].

ACKNOWLEDGMENT

The preferred spelling of the word “acknowledgment” in American English is without an “e” after the “g.” Use the singular heading even if you have many acknowledgments. Avoid expressions such as “One of us (S.B.A.) would like to thank” Instead, write “F. A. Author thanks” In most cases, sponsor and financial support acknowledgments are placed in the unnumbered footnote on the first page, not here.

REFERENCES

- [1] A. Martin-Lopez, S. Segura, and A. Ruiz-Cortés, “Online testing of RESTful APIs: Promises and challenges,” in *Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2022, pp. 408–420.
- [2] P. L. Ventre, M. M. Tajiki, S. Salsano, and C. Filsfils, “SDN architecture and southbound APIs for IPv6 segment routing enabled wide area networks,” *IEEE Trans. Netw. Service Manag.*, vol. 15, pp. 1378–1392, 2018.
- [3] X. Zhu, G. She, B. Xue, Y. Zhang, Y. Zhang, X. K. Zou, X. Duan, P. He, A. Krishnamurthy, and M. Lentz, “Dissecting overheads of service mesh sidecars,” in *Proc. 2023 ACM Symp. Cloud Comput.*, 2023, pp. 142–157.
- [4] L. Guo and J. Y. B. Lee, “Stateful-TCP—a new approach to accelerate TCP slow-start,” *IEEE Access*, vol. 8, pp. 195955–195970, 2020.
- [5] T. Pinheiro, M. Mialaret, P. Pereira, L. Lins, D. Silva, J. Dantas, and P. Maciel, “Performance modeling of microservices with circuit breakers using stochastic Petri nets,” in *Proc. 2024 IEEE Int. Syst. Conf. (SysCon)*, 2024, pp. 1–8.
- [6] B. D’Auria, I. Adan, R. Bekker, and V. Kulkarni, “An M/M/c queue with queueing-time dependent service rates,” *Eur. J. Oper. Res.*, vol. 299, pp. 566–579, 2021.
- [7] J. D. C. Little, “OR forum—Little’s law as viewed on its 50th anniversary,” *Oper. Res.*, vol. 59, no. 3, pp. 536–549, 2011.
- [8] Unicorn, “Unicorn: Serve Python on the Web.” [Online]. Available: <https://unicorn.org/>. [Accessed: May 14, 2026].
- [9] P. J. Eby, “PEP 333 – Python Web Server Gateway Interface v1.0,” Sep. 27, 2010. [Online]. Available: <https://peps.python.org/pep-0333/>. [Accessed: Apr. 23, 2026].
- [10] G. Tene, “A HTTP benchmarking tool based mostly on wrk,” Jun. 28, 2016. [Online]. Available: <https://github.com/giltene/wrk2>. [Accessed: May 14, 2026].
- [11] B. Djurkovic, “gRPC benchmarking and load testing tool,” Dec. 7, 2025. [Online]. Available: <https://github.com/bojand/ghz>. [Accessed: May 14, 2026].
- [12] H. C. Tijms, *A first course in stochastic models*, vol. 2, Wiley New York, 2003.
- [13] P. Banerjee, S. Roy, A. Sinha, M. Hassan, S. Burje, A. Agrawal, A. K. Bairagi, S. Alshathri, and W. El-shafai, “MTD-DHJS: Makespan-Optimized Task Scheduling Algorithm for Cloud Computing With Dynamic Computational Time Prediction,” *IEEE Access*, vol. 11, pp. 105578–105618, 2023.
- [14] G. O. Young, “Synthetic structure of industrial plastics,” in *Plastics*, 2nd ed., vol. 3, J. Peters, Ed. New York, NY, USA: McGraw-Hill, 1964, pp. 15–64.
- [15] W.-K. Chen, *Linear Networks and Systems*. Belmont, CA, USA: Wadsworth, 1993, pp. 123–135.
- [16] J. U. Duncombe, “Infrared navigation—Part I: An assessment of feasibility,” *IEEE Trans. Electron Devices*, vol. ED-11, no. 1, pp. 34–39, Jan. 1959, 10.1109/TED.1964.2628402.
- [17] E. P. Wigner, “Theory of traveling-wave optical laser,” *Phys. Rev.*, vol. 134, pp. A635–A646, Dec. 1965.
- [18] E. H. Miller, “A note on reflector arrays,” *IEEE Trans. Antennas Propagat.*, to be published.
- [19] E. E. Reber, R. L. Michell, and C. J. Carter, “Oxygen absorption in the earth’s atmosphere,” Aerospace Corp., Los Angeles, CA, USA, Tech. Rep. TR-0200 (4230-46)-3, Nov. 1988.
- [20] J. H. Davis and J. R. Cogdell, “Calibration program for the 16-foot antenna,” Elect. Eng. Res. Lab., Univ. Texas, Austin, TX, USA, Tech. Memo. NGL-006-69-3, Nov. 15, 1987.
- [21] *Transmission Systems for Communications*, 3rd ed., Western Electric Co., Winston-Salem, NC, USA, 1985, pp. 44–60.
- [22] *Motorola Semiconductor Data Manual*, Motorola Semiconductor Products Inc., Phoenix, AZ, USA, 1989.
- [23] G. O. Young, “Synthetic structure of industrial plastics,” in *Plastics*, vol. 3, Polymers of Hexadromicon, J. Peters, Ed., 2nd ed. New York, NY, USA: McGraw-Hill, 1964, pp. 15–64. [Online]. Available: <http://www.bookref.com>.
- [24] *The Founders’ Constitution*, Philip B. Kurland and Ralph Lerner, eds., Chicago, IL, USA: Univ. Chicago Press, 1987. [Online]. Available: <http://press-pubs.uchicago.edu/founders/>
- [25] The Terahertz Wave eBook. ZOmega Terahertz Corp., 2014. [Online]. Available: http://dl.z-thz.com/eBook/zomegaebookpdf_1206_sr.pdf. Accessed on: May 19, 2014.
- [26] Philip B. Kurland and Ralph Lerner, eds., *The Founders’ Constitution*. Chicago, IL, USA: Univ. of Chicago Press, 1987, Accessed on: Feb. 28, 2010. [Online] Available: <http://press-pubs.uchicago.edu/founders/>
- [27] J. S. Turner, “New directions in communications,” *IEEE J. Sel. Areas Commun.*, vol. 13, no. 1, pp. 11–23, Jan. 1995.
- [28] W. P. Risk, G. S. Kino, and H. J. Shaw, “Fiber-optic frequency shifter using a surface acoustic wave incident at an oblique angle,” *Opt. Lett.*, vol. 11, no. 2, pp. 115–117, Feb. 1986.
- [29] P. Kopyt et al., “Electric properties of graphene-based conductive layers from DC up to terahertz range,” *IEEE THz Sci. Technol.*, to be published. DOI: 10.1109/THZ.2016.2544142.
- [30] PROCESS Corporation, Boston, MA, USA. Intranets: Internet technologies deployed behind the firewall for corporate productivity. Presented at INET96 Annual Meeting. [Online]. Available: <http://home.process.com/Intranets/wp2.htm>
- [31] R. J. Hijmans and J. van Etten, “Raster: Geographic analysis and modeling with raster data,” R Package Version 2.0-12, Jan. 12, 2012. [Online]. Available: <http://CRAN.R-project.org/package=raster>
- [32] Teralyzer. Lytera UG, Kirchhain, Germany [Online]. Available: http://www.lytera.de/Terahertz_THz_Spectroscopy.php?id=home, Accessed on: Jun. 5, 2014.
- [33] U.S. House. 102nd Congress, 1st Session. (1991, Jan. 11). *H. Con. Res. 1, Sense of the Congress on Approval of Military Action*. [Online]. Available: LEXIS Library: GENFED File: BILLS
- [34] Musical toothbrush with mirror, by L.M.R. Brooks. (1992, May 19). Patent D 326 189 [Online]. Available: NEXIS Library: LEXPAT File: DES
- [35] D. B. Payne and J. R. Stern, “Wavelength-switched passively coupled single-mode optical network,” in *Proc. IOOC-ECOC*, Boston, MA, USA, 1985, pp. 585–590.
- [36] D. Ebehard and E. Voges, “Digital single sideband detection for interferometric sensors,” presented at the 2nd Int. Conf. Optical Fiber Sensors, Stuttgart, Germany, Jan. 2–5, 1984.
- [37] G. Brandli and M. Dick, “Alternating current fed power supply,” U.S. Patent 4 084 217, Nov. 4, 1978.
- [38] J. O. Williams, “Narrow-band analyzer,” Ph.D. dissertation, Dept. Elect. Eng., Harvard Univ., Cambridge, MA, USA, 1993.
- [39] N. Kawasaki, “Parametric study of thermal and chemical nonequilibrium nozzle flow,” M.S. thesis, Dept. Electron. Eng., Osaka Univ., Osaka, Japan, 1993.
- [40] A. Harrison, private communication, May 1995.
- [41] B. Smith, “An approach to graphs of linear forms,” unpublished.
- [42] A. Brahms, “Representation error for real numbers in binary computer arithmetic,” IEEE Computer Group Repository, Paper R-67-85.
- [43] IEEE Criteria for Class IE Electric Systems, IEEE Standard 308, 1969.
- [44] Letter Symbols for Quantities, ANSI Standard Y10.5-1968.
- [45] R. Fardel, M. Nagel, F. Nuesch, T. Lippert, and A. Wokaun, “Fabrication of organic light emitting diode pixels by laser-assisted forward transfer,” *Appl. Phys. Lett.*, vol. 91, no. 6, Aug. 2007, Art. no. 061103.
- [46] J. Zhang and N. Tansu, “Optical gain and laser characteristics of InGaN quantum wells on ternary InGaN substrates,” *IEEE Photon. J.*, vol. 5, no. 2, Apr. 2013, Art. no. 2600111
- [47] S. Azodolmolky et al., Experimental demonstration of an impairment aware network planning and operation tool for transparent/translucent optical networks,” *J. Lightw. Technol.*, vol. 29, no. 4, pp. 439–448, Sep. 2011.



FIRST A. AUTHOR received the B.S. and M.S. degrees in aerospace engineering from the University of Virginia, Charlottesville, in 2001 and the Ph.D. degree in mechanical engineering from Drexel University, Philadelphia, PA, in 2008.

From 2001 to 2004, he was a Research Assistant with the Princeton Plasma Physics Laboratory. Since 2009, he has been an Assistant Professor with the Mechanical Engineering Department, Texas A&M University, College Station. He is the

author of three books, more than 150 articles, and more than 70 inventions. His research interests include high-pressure and high-density nonthermal plasma discharge processes and applications, microscale plasma discharges, discharges in liquids, spectroscopic diagnostics, plasma propulsion, and innovation plasma applications. He is an Associate Editor of the journal *Earth, Moon, Planets*, and holds two patents.

Dr. Author was a recipient of the International Association of Geomagnetism and Aeronomy Young Scientist Award for Excellence in 2008, and the IEEE Electromagnetic Compatibility Society Best Symposium Paper Award in 2011.

THIRD C. AUTHOR, JR. (M'87) received the B.S. degree in mechanical engineering from National Chung Cheng University, Chiayi, Taiwan, in 2004 and the M.S. degree in mechanical engineering from National Tsing Hua University, Hsinchu, Taiwan, in 2006. He is currently pursuing the Ph.D. degree in mechanical engineering at Texas A&M University, College Station, TX, USA.

From 2008 to 2009, he was a Research Assistant with the Institute of Physics, Academia Sinica, Taipei, Taiwan. His research interest includes the development of surface processing and biological/medical treatment techniques using nonthermal atmospheric pressure plasmas, fundamental study of plasma sources, and fabrication of micro- or nanostructured surfaces.

Mr. Author's awards and honors include the Frew Fellowship (Australian Academy of Science), the I. I. Rabi Prize (APS), the European Frequency and Time Forum Award, the Carl Zeiss Research Award, the William F. Meggers Award and the Adolph Lomb Medal (OSA).

...



SECOND B. AUTHOR (M'76–SM'81–F'87) and all authors may include biographies. Biographies are often not included in conference-related papers. This author became a Member (M) of IEEE in 1976, a Senior Member (SM) in 1981, and a Fellow (F) in 1987. The first paragraph may contain a place and/or date of birth (list place, then date). Next, the author's educational background is listed. The degrees should be listed with type of degree in what field, which institution, city, state, and country, and year the degree was earned. The author's major field of study should be lower-cased.

The second paragraph uses the pronoun of the person (he or she) and not the author's last name. It lists military and work experience, including summer and fellowship jobs. Job titles are capitalized. The current job must have a location; previous positions may be listed without one. Information concerning previous publications may be included. Try not to list more than three books or published articles. The format for listing publishers of a book within the biography is: title of book (publisher name, year) similar to a reference. Current and previous research interests end the paragraph.

The third paragraph begins with the author's title and last name (e.g., Dr. Smith, Prof. Jones, Mr. Kajor, Ms. Hunter). List any memberships in professional societies other than the IEEE. Finally, list any awards and work for IEEE committees and publications. If a photograph is provided, it should be of good quality, and professional-looking. Following are two examples of an author's biography.